1、Significant earthquakes since 2150 B.C.

1.1

思路：

将所需包和数据导入，对所需数据列进行清洗，将异常数据转为 NaN 防止报错，在根据国家分组求和，输出前 20 个国家名称及死亡人数

代码：

```
import numpy as np
import pandas as pd
import xarray as xr
import netCDF4
from matplotlib import pyplot as plt
import netCDF4
import xarray as xr
```

```
#数据读取
df = pd.read_csv("Sig_Eqs.tsv", sep="\t")
```

```
#1.1
#数据清洗，提高鲁棒性
df["Total Deaths"] = pd.to_numeric(df["Total Deaths"], errors="coerce")
res = df.groupby("Country")["Total Deaths"].sum().sort_values(ascending=False).head(20)
print(res)
```

结果：

```
Country
CHINA           2106526.0
TURKEY          1008867.0
IRAN             761656.0
SYRIA            488326.0
ITALY            417297.0
HAITI            323782.0
JAPAN            319510.0
AZERBAIJAN       319251.0
INDONESIA        283485.0
ARMENIA          175000.0
PAKISTAN         143714.0
ECUADOR          134445.0
TURKMENISTAN     110412.0
PERU              97009.0
GREECE            83658.0
PORTUGAL          82572.0
IRAQ              70200.0
CHILE             70175.0
INDIA             63413.0
ISRAEL            58368.0
Name: Total Deaths, dtype: float64
```

1.2

思路：

首先清洗数据，再根据“MS”列信息进行计算，保留震级大于 3.0 的数据，随后根据年份“Year”分组，再使用“size”统计每年有多少条信息，然后按索引（即年份）输出，最后对图表进行了格式上的优化以便更好观察趋势

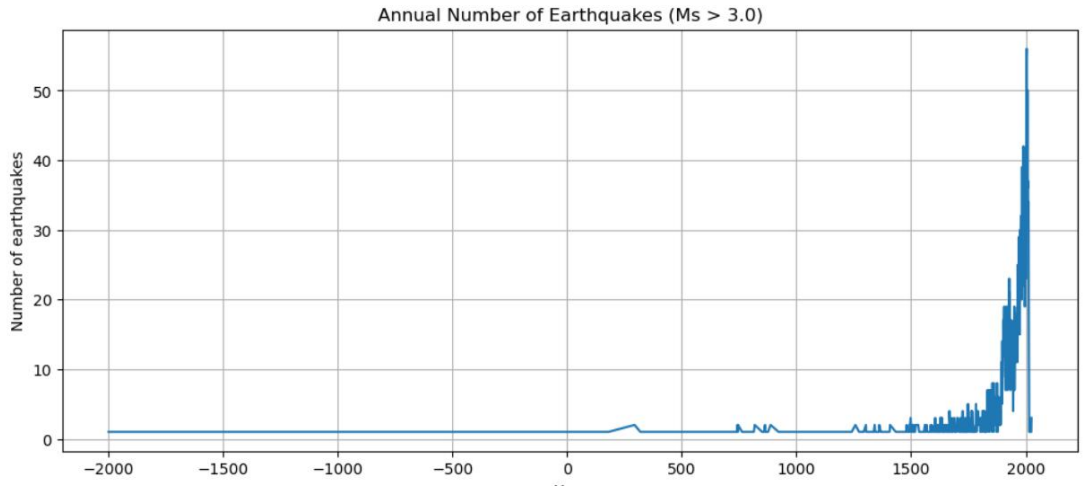
代码：

```
#1.2
#数据清洗删除年份或震级缺失的数据
df3 = df.dropna(subset=["Year", "Ms"])
df3 = df3[df3["Ms"] > 3.0]

# 根据结果优化了表格尺寸增加了表格标题等
df3.groupby("Year").size().sort_index().plot(
    figsize=(12, 5), xlabel="Year", ylabel="Number of earthquakes",
    title="Annual Number of Earthquakes (Ms > 3.0)", grid=True
)
plt.show()

#根据结果可观测到地震记录整体呈上升趋势尤其是1500年之后接近2000年时涨幅非常明显，可能是由于接近现代地震监测技术的上升和数据统计
```

结果：



可以观察到 1000 年前地震记录几乎为 0，1250 年前后对地震的记录开始增多，1500 年之后呈指数型增长，在 2000 年出现了下降，但并不一定代表实际地震的趋势，早年几乎没有地震可能是由于数据的缺失，后续观测手段提高对数据的记录更加完善，而 2000 年后折线下降是由于表格时间尺度大，短期地震次数少

1.3

思路：

定义一个函数，输入值代表某个国家所有地震记录（由于后续调用时用“Country”分组），这样就可以通过统计多少条记录来得到地震总数；

增加.empty()筛选没有有效震级的记录防止报错；

再使用.max()寻找最大震级并读取第一条记录，随后输出对应地震的日期，如果

年月日信息完整就输出完整日期，否则输出年份（考虑到格式问题，简化只输出年份），最后将结果输出为三列，最后调用函数降序输出结果

代码：

```
#1.3
# 定义函数：统计地震总数，并返回最大地震的日期和地点
def CountEq_LargestEq(data):
    # 统计该国家的地震总数
    total = len(data)
    # 找出有震级记录的地震
    valid = data.dropna(subset=["Mag"])

    # 如果该国家没有有效震级记录则返回结果防止报错
    if valid.empty:
        return pd.Series({
            "Total Earthquakes": total,
            "Largest Earthquake Date": "Unknown",
            "Largest Earthquake Location": "Unknown"
        })

    # 找到最大震级
    max_ms = valid["Mag"].max()
    largest = valid.loc[valid["Mag"] == max_ms].iloc[0]

    # 输出最大地震的日期
    if pd.notna(largest["Mo"]) and pd.notna(largest["Dy"]):
        date = (
            f'{int(largest["Year"])}-'
            f'{int(largest["Mo"]):02d}-'
            f'{int(largest["Dy"]):02d}'
        )
    #如果月日信息缺失
    else:
        date = str(int(largest["Year"]))

    return pd.Series({
        "Total Earthquakes": total,
        "Largest Earthquake Date": date,
        "Largest Earthquake Location": largest["Location Name"]
    })

#调用函数并降序输出结果
result = df.groupby("Country").apply(
    CountEq_LargestEq,
    include_groups=False
).sort_values("Total Earthquakes", ascending=False)

display(result)
```

结果：

Country	Total Earthquakes	Largest Earthquake Date	Largest Earthquake Location
CHINA	626	1668-07-25	CHINA: SHANDONG PROVINCE
JAPAN	429	2011-03-11	JAPAN: HONSHU
INDONESIA	427	2004-12-26	INDONESIA: SUMATRA: ACEH: OFF WEST COAST
IRAN	385	856-12-22	IRAN: DAMGHAN, QUMIS
TURKEY	359	1939-12-26	TURKEY: ERZINCAN
...	...	...	...
SRI LANKA	1	Unknown	Unknown
MONTSERRAT	1	Unknown	Unknown
PALAU	1	1914-10-23	MICRONESIA, FED. STATES OF: CAROLINE ISLANDS
SAINT VINCENT AND THE GRENADINES	1	Unknown	Unknown
ZAMBIA	1	2017-02-24	ZAMBIA: KAPUTA

162 rows × 3 columns

2、Air temperature in Shenzhen 1998-2022

筛选数据思路：

首先分析本次要解决的问题有原始数据可用列，观察到“DATE”“TMP”为主要数据源，根据用户指南可知“TMP”用“，”分割气温和质量标记，故在数据清洗时将其分别存入两个新列中；并且根据用户指南“SCALING FACTOR:10”可知气温数据要进行除以十的操作才为真实值；

随后进行有效数据筛选，根据用户指南，我选择质量标记为“1”和“5”（代表通过全部质量检查，无可疑或错误数据），同时满足日期有效、温度代码有效且不等于 9999（缺失值）的数据，满足以上条件的计入计算范围；

最后限定研究时间并按月计算平均气温，可视化输出。

代码：

```
#2深圳气温

#筛选数据方法在报告中详解

# 读取气象数据
weather_data = pd.read_csv(
    "Baoan_Weather_1998_2022.csv",
    low_memory=False
)

# 清洗时间数据
weather_data["DATE"] = pd.to_datetime(
    weather_data["DATE"], errors="coerce"
)

# 拆分TMP字段，分别提取气温值和质量标记
temperature_parts = (
    weather_data["TMP"]
    .astype("string")
    .str.split(",", expand=True)
)

weather_data["temperature_code"] = pd.to_numeric(
    temperature_parts[0], errors="coerce"
)

weather_data["quality_flag"] = temperature_parts[1]

# 将气温原始值转换为摄氏度
weather_data["air_temp_celsius"] = (
    weather_data["temperature_code"] / 10
)

# 筛选有效气温记录
clean_weather = weather_data[
    weather_data["DATE"].notna()
    & weather_data["temperature_code"].notna()
    & (weather_data["temperature_code"] != 9999)
    & weather_data["quality_flag"].isin(["1", "5"])
].copy()
```

```python
# 限定研究时间
clean_weather = clean_weather[
    (clean_weather["DATE"] >= "1998-01-01")
    & (clean_weather["DATE"] < "2023-01-01")
]

# 计算每月平均气温
clean_weather = clean_weather.set_index("DATE")
monthly_average = (
    clean_weather["air_temp_celsius"]
    .resample("MS")
    .mean()
)

# 绘制月平均气温时间序列
plt.figure(figsize=(14, 6))

plt.plot(
    monthly_average.index,
    monthly_average.values,
    linewidth=1
)

plt.title("Monthly Mean Air Temperature in Shenzhen (1998-2022)")
plt.xlabel("Observation Time")
plt.ylabel("Monthly Mean Air Temperature (°C)")

plt.xlim(
    pd.Timestamp("1998-01-01"),
    pd.Timestamp("2022-12-31")
)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```python
#接第二题可视化

# 使用循环每年绘制一张月平均气温图
for year in range(1998, 2023):

    # 提取当前年份的月平均气温
    yearly_average = monthly_average[
        monthly_average.index.year == year
    ]

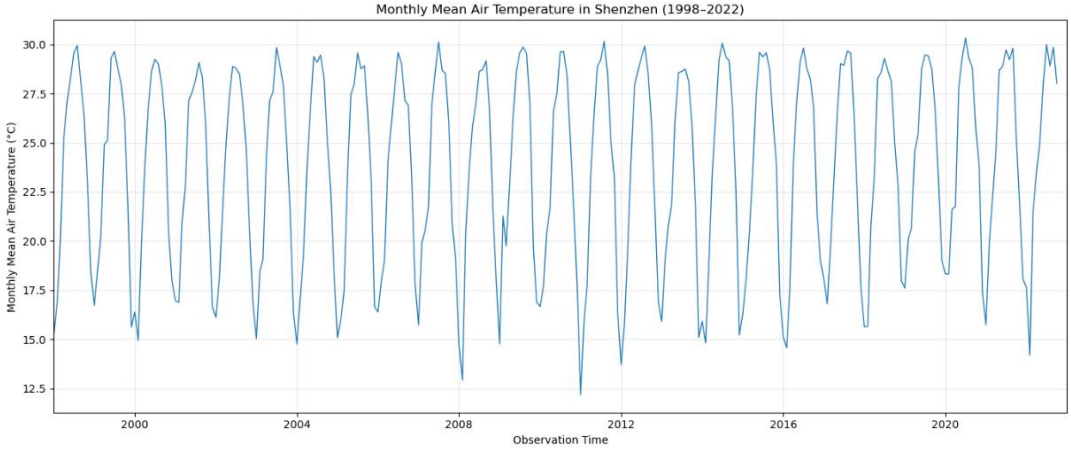
    # 创建当前年份的图像
    plt.figure(figsize=(10, 5))

    # 绘制月份与月平均气温的关系
    plt.plot(
        yearly_average.index.month,
        yearly_average.values,
        marker="o",
        linewidth=1.5
    )

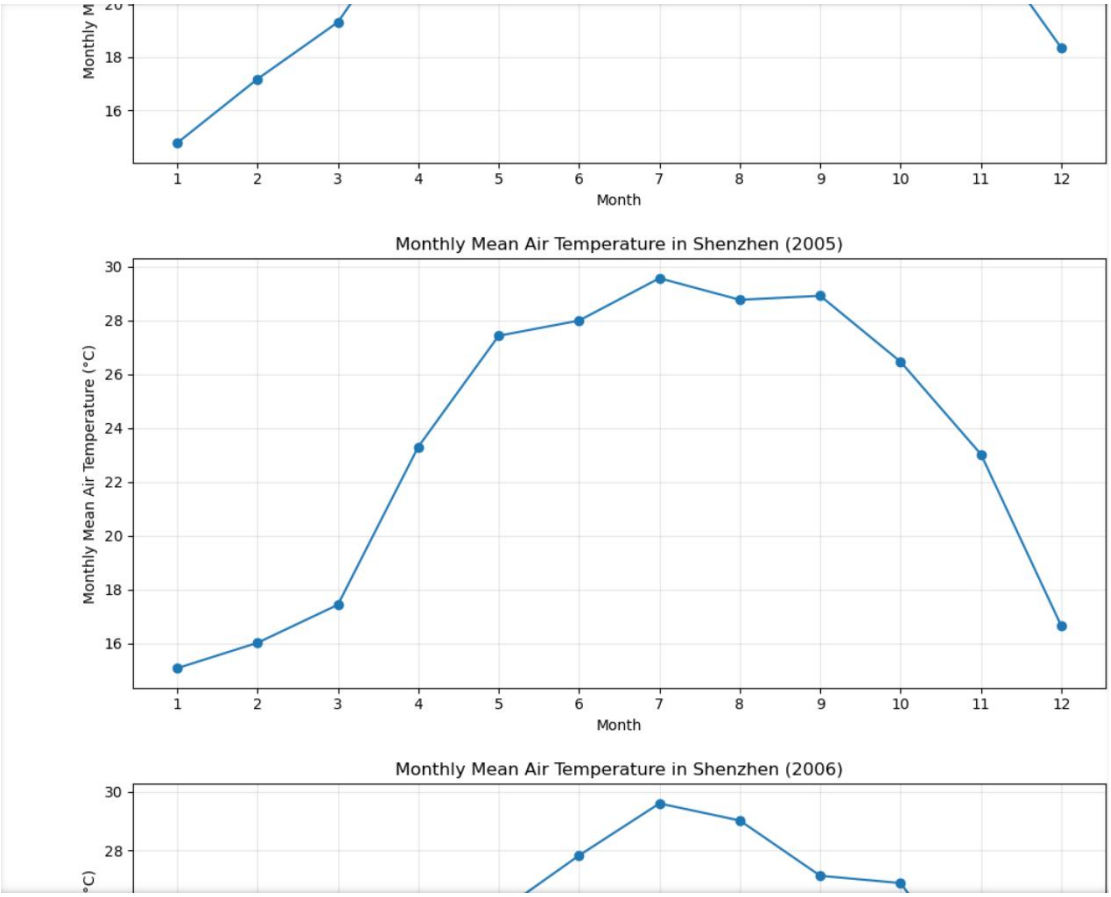
    # 设置标题和坐标轴
    plt.title(f"Monthly Mean Air Temperature in Shenzhen ({year})")
    plt.xlabel("Month")
    plt.ylabel("Monthly Mean Air Temperature (°C)")

    # 设置横轴显示 1 到 12 月并添加网格
    plt.xticks(range(1, 13))
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```

结果：



从折线图可观察到，从 1998-2022 二十多年来年均气温整体保持在较稳定的状态，每年最高月均气温稳定在 30 摄氏度上下，存在小幅增长或下降，每年月均最低气温浮动比较大，最低低于 12.5 摄氏度，为了更细致进行观察，增加对每年的月均气温制图输出。



观察每年的月均气温折线图可知，月均最低气温一般出现在 1、2 月份，3-5 月一般是升温最明显的时段，每年的月均气温最高值一般出现在 7、8 月份，随后开始下降，但不是每年稳定上述趋势，需要分析。

3、Global collection of hurricanes

3.1

思路：
跟前面题目一样先读取数据并清洗，随后根据 SID 进行.groupby()分组，取风速最大的前十名台风，输出名称和风速
代码：

```
# 3.1
# 读取数据并清洗
hc = pd.read_csv(
    "ibtracs.ALL.list.v04r00.csv",
    usecols=range(17),
    skiprows=[1, 2],
    parse_dates=['ISO_TIME'],
    na_values=['NOT_NAMED', 'NAME'],
    low_memory=False
)

# 将 WMO_WIND 转换为数值类型
hc['WMO_WIND'] = pd.to_numeric(hc['WMO_WIND'], errors='coerce')

# 按 SID 分组，获取最大风速与台风名，并筛选排名前 10 的大台风
top_10 = (
    hc.groupby('SID')
    .agg({'NAME': 'first', 'WMO_WIND': 'max'})
    .sort_values('WMO_WIND', ascending=False)
    .head(10)
)

print(top_10[['NAME', 'WMO_WIND']])
```

结果：

SID	NAME	WMO_WIND
2015293N13266	PATRICIA	185.0
1980214N11330	ALLEN	165.0
2019236N10314	DORIAN	160.0
1988253N12306	GILBERT	160.0
2005289N18282	WILMA	160.0
1935241N23291	NOT_NAMED	160.0
1997253N12255	LINDA	160.0
2017242N16333	IRMA	155.0
1998295N12284	MITCH	155.0
2009288N07267	RICK	155.0

3.2

思路：
同 3.1，只是将列举数量改为 20
代码：

```
#3.2
# 列出风速最强的前 20 个台风及其风速
top_20_winds = (
    hc.groupby('SID')
    .agg({'NAME': 'first', 'WMO_WIND': 'max'})
    .sort_values('WMO_WIND', ascending=False)
    .head(20)
)

print(top_20_winds[['NAME', 'WMO_WIND']])
```

结果：

SID	NAME	WMO_WIND
2015293N13266	PATRICIA	185.0
1980214N11330	ALLEN	165.0
2019236N10314	DORIAN	160.0
1988253N12306	GILBERT	160.0
2005289N18282	WILMA	160.0
1935241N23291	NOT_NAMED	160.0
1997253N12255	LINDA	160.0
2017242N16333	IRMA	155.0
1998295N12284	MITCH	155.0
2009288N07267	RICK	155.0
2005261N21290	RITA	155.0
1955265N13307	JANET	150.0
1992230N11325	ANDREW	150.0
2016041S14170	WINSTON	150.0
1977242N27272	ANITA	150.0
2017260N12310	MARIA	150.0
2005236N23285	KATRINA	150.0
1959249N19181	PATSY	150.0
1979238N12324	DAVID	150.0
1994222N11267	JOHN	150.0

3.3

思路：
使用.value_count()函数计算“Basin”各数据点的数量，跳过不包含有效数据的行
代码：

```
#3.3
# 过滤掉缺失值（NaN）以及只包含空格的行
basin_counts = hc['BASIN'].replace(r'^\s*$', pd.NA, regex=True).dropna().value_counts()
print(basin_counts)
```

结果：

```
BASIN
WP    236576
SI    160668
SP     67119
EP     62412
NI     55402
SA       119
Name: count, dtype: int64
```

3.4

思路：

读取经纬度数据并转换为数值，使用.hexbin()绘制六边形地图，没有数据的位置用“mincnt=1”语句保持空白

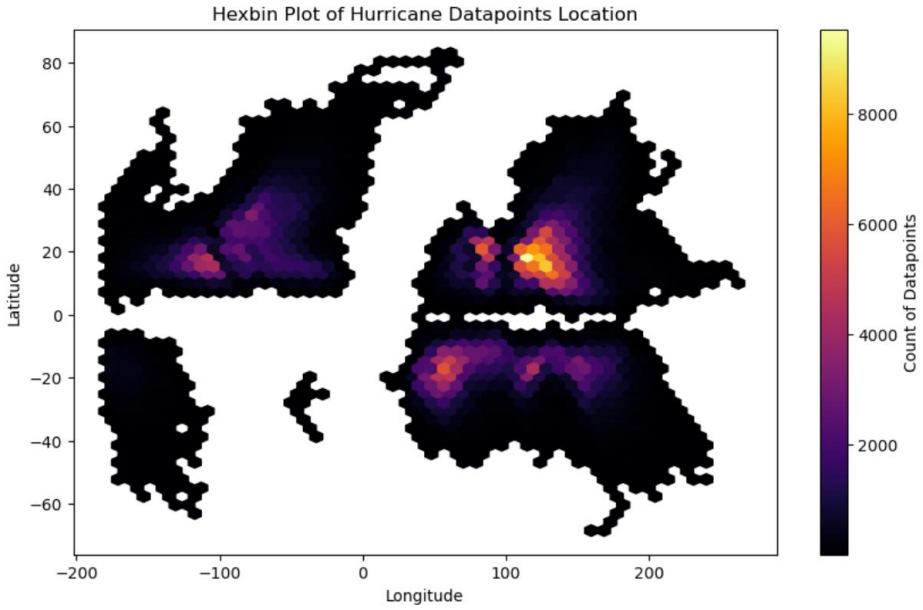
代码：

```
#3.4
# 提取经纬度数据并转换为数值类型
lat_data = pd.to_numeric(hc['LAT'], errors='coerce')
lon_data = pd.to_numeric(hc['LON'], errors='coerce')

# 创建画布并绘制 hexbin图，使用mincnt让数值为0的区域保持空白
plt.figure(figsize=(10, 6))
hb = plt.hexbin(lon_data, lat_data, gridsize=50, cmap='inferno', mincnt=1)

# 添加颜色条和标题标签
plt.colorbar(hb, label='Count of Datapoints')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('Hexbin Plot of Hurricane Datapoints Location')
plt.show()
```

结果：



3.5

思路：

根据“NAME”筛选所需的台风数据，提取时间年份为“2018”的数据，将经纬度数据进行转换，再使用.plot()制图，自定义绘制方案，添加 XY 轴美化制图

代码：

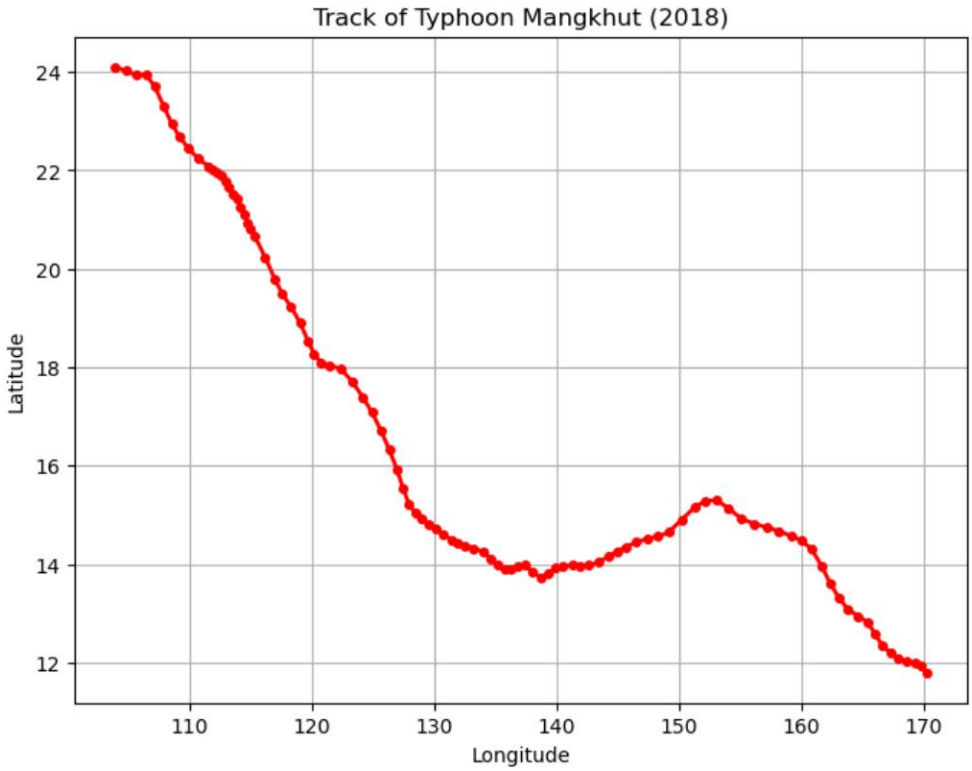
```
# 3.5
# 过滤出名称为 MANGKHUT 且年份在 2018 年的数据
mangkhut_df = hc[hc['NAME'] == 'MANGKHUT']
mangkhut_2018 = mangkhut_df[mangkhut_df['ISO_TIME'].dt.year == 2018]

# 转换经纬度为数值
m_lat = pd.to_numeric(mangkhut_2018['LAT'], errors='coerce')
m_lon = pd.to_numeric(mangkhut_2018['LON'], errors='coerce')

# 绘制轨迹图
plt.figure(figsize=(8, 6))
plt.plot(m_lon, m_lat, marker='o', color='red', linestyle='-', linewidth=2, markersize=4)

plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('Track of Typhoon Mangkhut (2018)')
plt.grid(True)
plt.show()
```

结果：



3.6

思路：

提取时间列前四个字符（年份），筛选年份在 1970 及以后，且“Basin”列包含‘WP’和‘EP’的数据，复制到新的变量“hc_filtered”中，打印过滤前后数据的行数验证筛选是否有效进行

代码：

```
# 3.6
# 直接使用 .dt 提取年份
years = hc['ISO_TIME'].dt.year

# 筛选：时间 >= 1970 且 BASIN 在 ['WP', 'EP'] 中
hc_filtered = hc[
    (years >= 1970) &
    (hc['BASIN'].isin(['WP', 'EP']))
].copy()

print(f"过滤前数据总行数: {len(hc)}")
print(f"过滤后数据总行数: {len(hc_filtered)}")
```

结果：

```
过滤前数据总行数: 707178
过滤后数据总行数: 176352
```

3.7

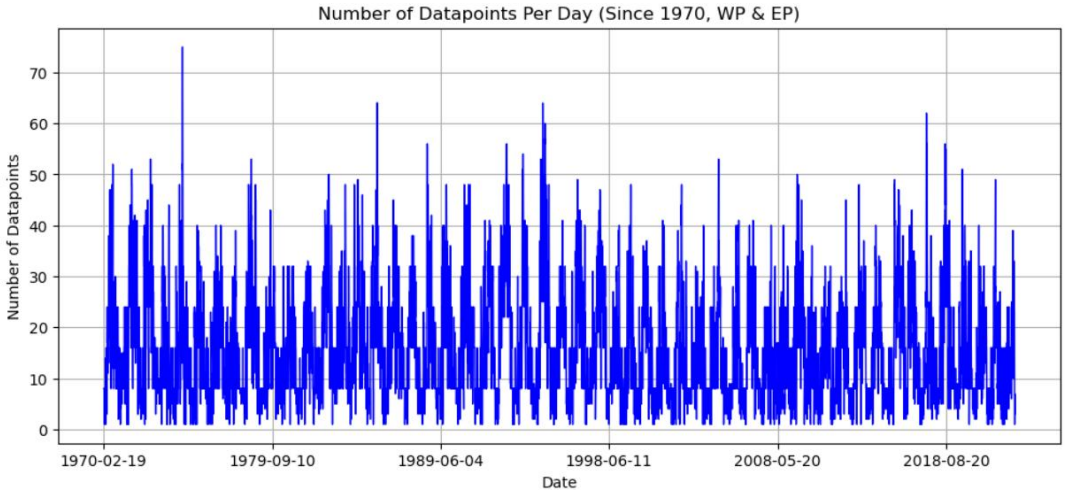
思路：
日期提取年月日部分，再根据日期统计点数，最后按时间排序
代码：

```
# 3.7
# 提取出日期部分 (YYYY-MM-DD)
dates = hc_filtered['ISO_TIME'].dt.strftime('%Y-%m-%d')
daily_counts = dates.value_counts().sort_index()

# 绘制每日数据点数量的时间序列折线图
plt.figure(figsize=(12, 5))
daily_counts.plot(kind='line', color='blue', linewidth=1)

plt.xlabel('Date')
plt.ylabel('Number of Datapoints')
plt.title('Number of Datapoints Per Day (Since 1970, WP & EP)')
plt.grid(True)
plt.show()
```

结果：



3.8

思路：
先将时间列转换为年月日的时间格式，随后使用 dt.dayofyear 将日期转为 1 到 365（或 366）的数字，在计算每天有多少数据点，最后制图展示
代码：

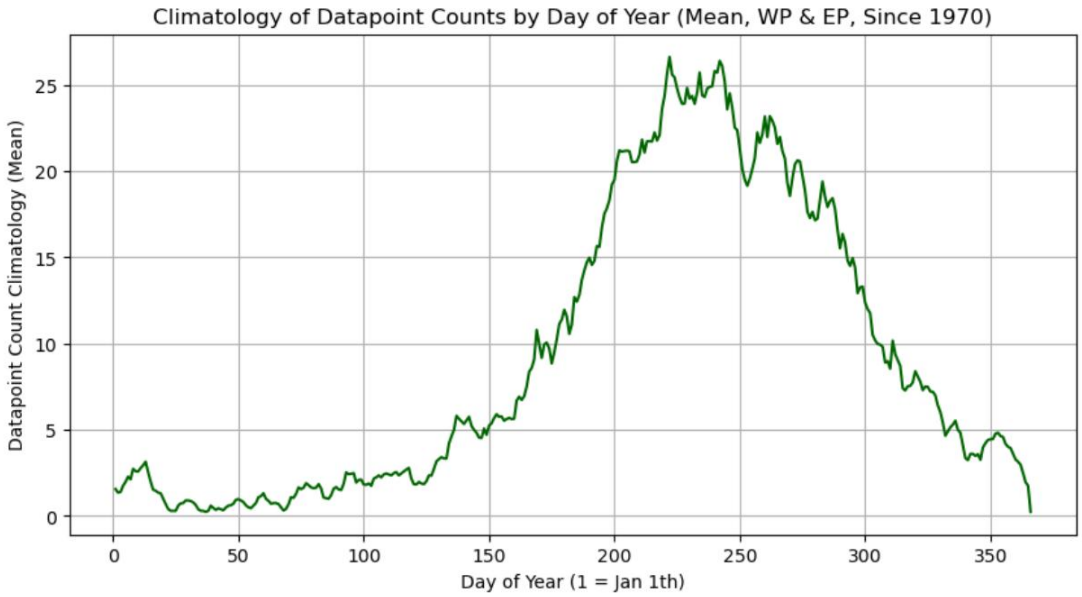
```python
# 3.8
# 确保 ISO_TIME 为 datetime 类型，并提取年份和一年中的第几天
hc_filtered['Year'] = hc_filtered['ISO_TIME'].dt.year
hc_filtered['DOY'] = hc_filtered['ISO_TIME'].dt.dayofyear

# 先按年份和 Day of Year 统计每年每一天的频次，然后求所有年份的平均值 (mean)
doy_yearly_counts = hc_filtered.groupby(['Year', 'DOY']).size().unstack(fill_value=0)
doy_climatology = doy_yearly_counts.mean(axis=0).sort_index()

# 绘制气候态分布图
plt.figure(figsize=(10, 5))
doy_climatology.plot(kind='line', color='darkgreen', linewidth=1.5)

plt.xlabel('Day of Year (1 = Jan 1th)')
plt.ylabel('Datapoint Count Climatology (Mean)')
plt.title('Climatology of Datapoint Counts by Day of Year (Mean, WP & EP, Since 1970)')
plt.grid(True)
plt.show()
```

结果：



3.9

思路：

计算每日实际的台风数据点数量，接着将日期转换为一年中的第几天，并利用 groupby().mean() 计算出历年同期（Day of Year）的平均气候态基准；最后用每日的实际计数减去对应的气候态均值，得到异常值，最后通过折线图可视化，添加 0 基准线作异常对比，与基准线差距越大异常程度越大

代码：

```python
# 3.9
# 获取每日实际数据点数量（通过 .dt.strftime 转换为日期字符串）
daily_counts = hc_filtered['ISO_TIME'].dt.strftime('%Y-%m-%d').value_counts().sort_index()

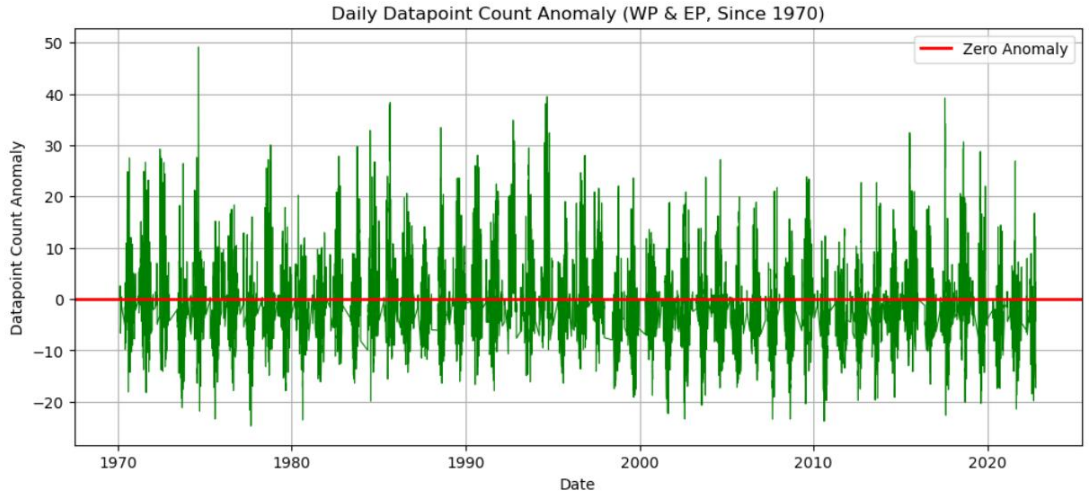
# 获取每一天的 Day of Year 数值
date_dt = pd.to_datetime(daily_counts.index)
doy_series = date_dt.dayofyear

# 计算历年同一天的平均值
doy_mean = daily_counts.groupby(doy_series).mean()
expected_counts = doy_series.map(doy_mean)

# 计算异常值 (实际值 - 均值)
anomaly = daily_counts - expected_counts

# 绘制异常值时间序列图
plt.figure(figsize=(12, 5))
plt.plot(date_dt, anomaly, color='green', linewidth=0.8)
# 添加一条 0 值线增强对比
plt.axhline(0, color='red', linewidth=2, label='Zero Anomaly')
plt.xlabel('Date')
plt.ylabel('Datapoint Count Anomaly')
plt.title('Daily Datapoint Count Anomaly (WP & EP, Since 1970)')
plt.legend()
plt.grid(True)
plt.show()
```

结果：



3.10

思路：

使用之前计算的异常值变量 anomaly 进行逐年求均值（重采样），为了制图时 X 轴的日期表达简洁对索引进行格式化，最后制作柱状图展示，并打印异常值最高和最低的前五个年份

代码：

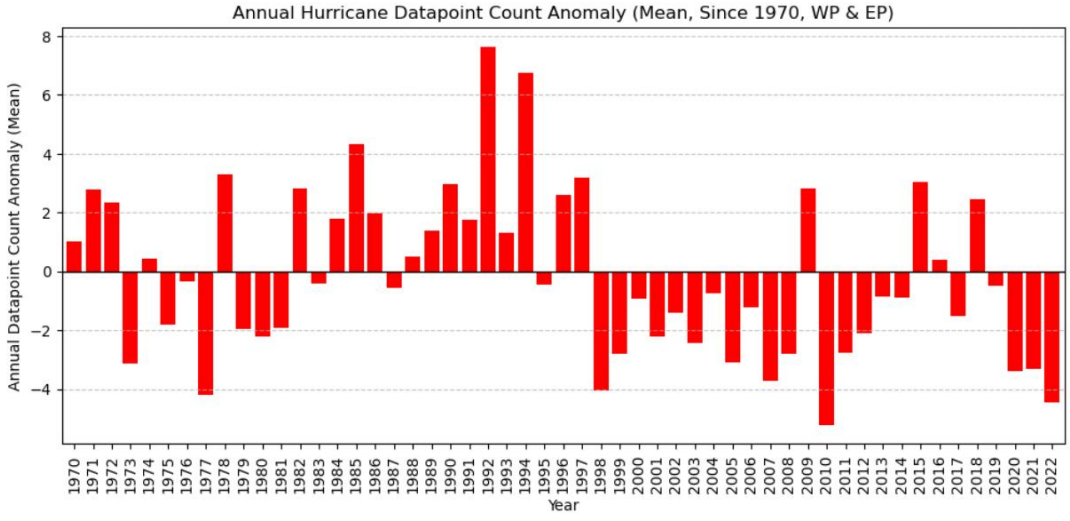
```python
# 3.10
# 将异常值信息格式统一并按年求平均值 (mean)
anomaly.index = pd.to_datetime(anomaly.index)
annual_anomaly = anomaly.resample('YE').mean()

# 将索引格式化为纯年份字符串
annual_anomaly.index = annual_anomaly.index.strftime('%Y')

# 3. 绘制年度异常值柱状图
plt.figure(figsize=(12, 5))
colors = ['red']
annual_anomaly.plot(kind='bar', color=colors, width=0.8)
plt.axhline(0, color='black', linewidth=1)
plt.xlabel('Year')
plt.ylabel('Annual Datapoint Count Anomaly (Mean)')
plt.title('Annual Hurricane Datapoint Count Anomaly (Mean, Since 1970, WP & EP)')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# 4. 打印排名前几位的异常年份
print("年度平均异常值最高（台风最活跃）的前5个年份:")
print(annual_anomaly.sort_values(ascending=False).head(5))
print("年度平均异常值最低（台风最沉寂）的前5个年份:")
print(annual_anomaly.sort_values(ascending=True).head(5))
```

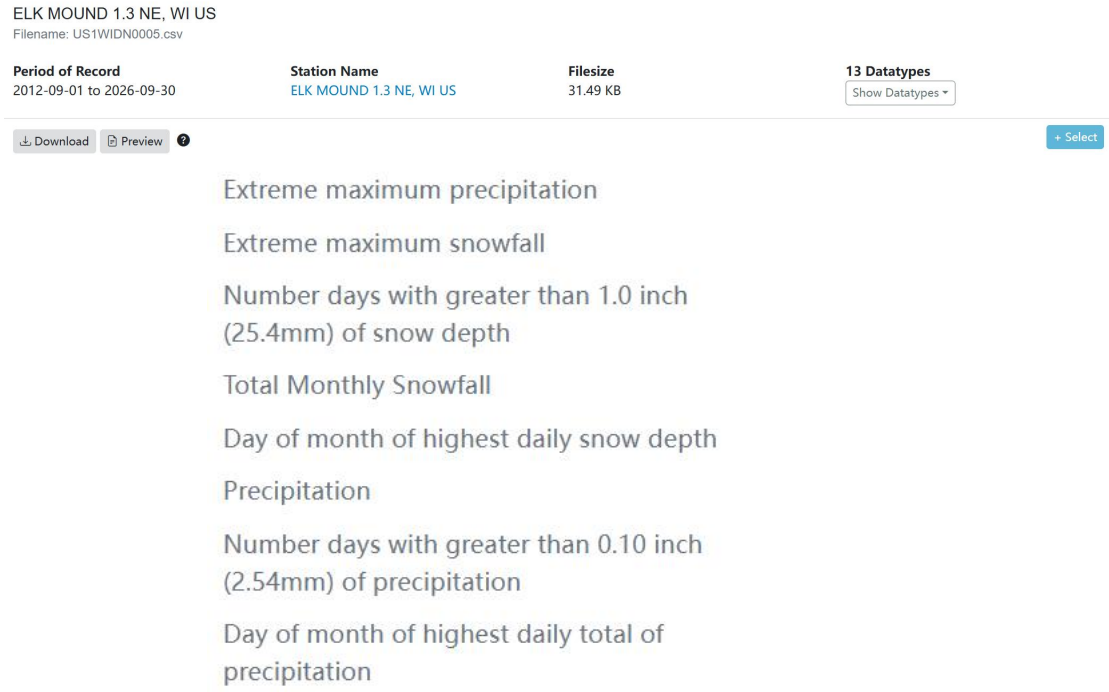
结果：



```
年度平均异常值最高（台风最活跃）的前5个年份:
ISO_TIME
1992    7.645086
1994    6.769985
1985    4.338109
1978    3.298956
1997    3.195031
dtype: float64
年度平均异常值最低（台风最沉寂）的前5个年份:
ISO_TIME
2010   -5.217217
2022   -4.456293
1977   -4.192921
1998   -4.035779
2007   -3.731739
dtype: float64
```

4、Explore a data set

我从国家环境信息中心下载了名为 US1WIDN0005.csv 的数据，时间序列从 2012.9.1 到 2026.9.30，包含 13 种数据类型，详情见下图



Number days with greater than 0.01 inch (0.25mm) of precipitation

Day of month of highest daily snowfall

Number days with greater than 1.00 inch (25.4mm) of precipitation

Number of days with greater than or equal to 1.0 inch (25.4mm) of snowfall

Extreme maximum snow depth

4.1

思路：

读取数据，将讲述、降雪和质量评价信息读入，考虑到总降水量信息最为完整，选取他作为后续查询主要数据，进行数据清洗时将 PRCP_ATTRIBUTES 列第三位代表数据质量的信息提取出来，筛除质量差的数据点，随后删除日期总降水量数据缺失的点，同时由于原始数据的 DATE 格式表达为“month-Year”，为防止计算出错将其转换为“Year-month”

代码：

```
# 4.1
df = pd.read_csv(
    "US1WIDN0005.csv",
    usecols=[
        "DATE",
        "PRCP", "PRCP_ATTRIBUTES",
        "SNOW", "SNOW_ATTRIBUTES",
        "EMXP", "EMXP_ATTRIBUTES",
        "EMSN", "EMSN_ATTRIBUTES",
        "EMSD", "EMSD_ATTRIBUTES",
        "DP01", "DP10", "DP1X",
        "DSNW", "DSND"
    ]
)
print("清洗前数据量：", len(df))

# 删除 PRCP_ATTRIBUTES 为空的行
df = df.dropna(subset=["PRCP_ATTRIBUTES"])

# 拆分属性字段，提取第三位质量标记，删除第三位有质量标记的行
flags = df["PRCP_ATTRIBUTES"].str.split(",", expand=True)
df = df[flags[2].fillna("").str.strip() == ""]

# 删除日期或降水量缺失的行
df = df.dropna(subset=["DATE", "PRCP"])

# 转换日期格式（原始数据为年-月）
df["DATE"] = pd.to_datetime(
    df["DATE"], format="%Y-%m", errors="coerce"
)
df = df.dropna(subset=["DATE"])

# 删除重复月份
df = df.drop_duplicates(subset=["DATE"])

# 日期展示为年-月形式
df["DATE"] = df["DATE"].dt.strftime("%Y-%m")

print("清洗后数据量：", len(df))
```

结果：

```
清洗前数据量：  140
清洗后数据量：  137
```

4.2

思路：

使用第一题清洗后的变量 PRCP，绘制月降水量时间序列图

代码：

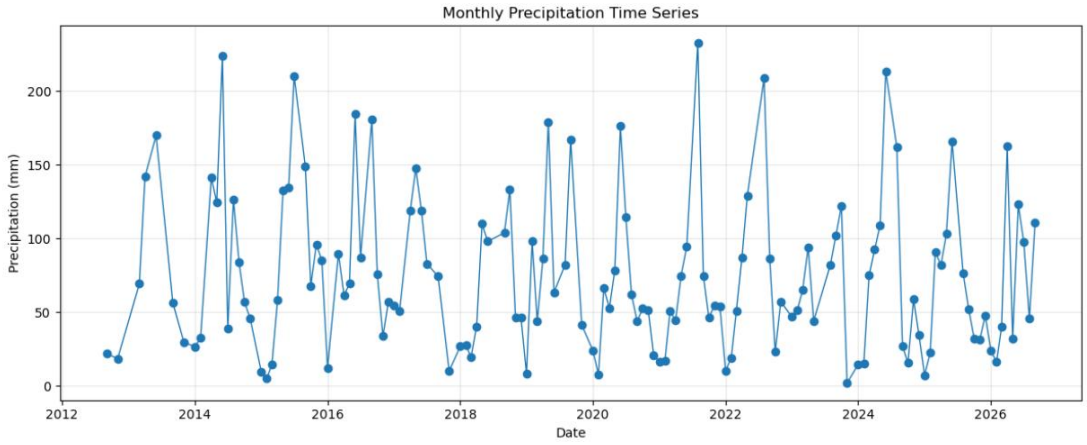
```python
#4.2
# 绘制时间序列图
df["DATE"] = pd.to_datetime(df["DATE"], format="%Y-%m")
plt.figure(figsize=(12, 5))

plt.plot(df["DATE"], df["PRCP"], marker="o", linewidth=1)

plt.title("Monthly Precipitation Time Series")
plt.xlabel("Date")
plt.ylabel("Precipitation (mm)")
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

结果：



4.3

思路：

选取常见的统计检查方式：观测数量、均值、中位数、第一四分位数（Q1）、第三四分位数（Q3）、四分位距（IQR）、最大值、最小值和标准差，并开展可视化

代码：

```python
#4.3
#数据准备
df["DATE"] = pd.to_datetime(df["DATE"], format="%Y-%m")
df["PRCP"] = pd.to_numeric(df["PRCP"], errors="coerce")
df = df.sort_values("DATE")
prcp = df["PRCP"]

# 2. 计算统计指标
Q1 = prcp.quantile(0.25)
Q2 = prcp.quantile(0.50)
Q3 = prcp.quantile(0.75)
IQR = Q3 - Q1

print("观测数量:", prcp.count())
print("均值:", round(prcp.mean(), 2))
print("中位数 Q2:", round(Q2, 2))
print("第一四分位数 Q1:", round(Q1, 2))
print("第三四分位数 Q3:", round(Q3, 2))
print("四分位距 IQR:", round(IQR, 2))
print("最小值:", round(prcp.min(), 2))
print("最大值:", round(prcp.max(), 2))
print("标准差:", round(prcp.std(), 2))

#可视化
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# 图1：降水量时间序列
axes[0, 0].plot(df["DATE"], prcp, linewidth=1)
axes[0, 0].axhline(
    prcp.mean(), linestyle="--", linewidth=1.5,
    label="Mean"
)
axes[0, 0].set_title("Monthly Precipitation Time Series")
axes[0, 0].set_xlabel("Date")
axes[0, 0].set_ylabel("Precipitation")
axes[0, 0].legend()
axes[0, 0].grid(alpha=0.3)
```

```
# 图2：总体箱线图
axes[0, 1].boxplot(prcp, vert=False, showmeans=True)
axes[0, 1].set_title("Precipitation Box Plot")
axes[0, 1].set_xlabel("Precipitation")
axes[0, 1].grid(axis="x", alpha=0.3)

# 图3：降水量频率分布
axes[1, 0].hist(prcp, bins=20, edgecolor="black", alpha=0.75)
axes[1, 0].axvline(
    prcp.mean(), linestyle="--", linewidth=1.5,
    label="Mean"
)
axes[1, 0].axvline(
    prcp.median(), linestyle=":", linewidth=1.5,
    label="Median"
)
axes[1, 0].set_title("Precipitation Frequency Distribution")
axes[1, 0].set_xlabel("Precipitation")
axes[1, 0].set_ylabel("Frequency")
axes[1, 0].legend()
axes[1, 0].grid(axis="y", alpha=0.3)

# 图4：不同月份的降水量箱线图
df["Month"] = df["DATE"].dt.month

monthly_data = [
    df.loc[df["Month"] == month, "PRCP"].dropna()
    for month in range(1, 13)
]

axes[1, 1].boxplot(
    monthly_data,
    tick_labels=[
        "Jan", "Feb", "Mar", "Apr", "May", "Jun",
        "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
    ],
    showmeans=True
)
axes[1, 1].set_title("Seasonal Precipitation Distribution")
axes[1, 1].set_xlabel("Month")
axes[1, 1].set_ylabel("Precipitation")
axes[1, 1].grid(axis="y", alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

结果：

观测数量: 137
均值: 74.94
中位数 Q2: 61.4
第一四分位数 Q1: 33.6
第三四分位数 Q3: 102.1
四分位距 IQR: 68.5
最小值: 1.9
最大值: 232.6
标准差: 52.93

